AFRILANG Stdlib

std/c/gameflood

Bibliothèque AFRILANG `std/c/gameflood` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : floodFill, conn

`import "std/c/gameflood"` · `use gameflood`

- `floodFill(grid list number, width number, start number, newVal number) ? list number`
- `connectedRegion(grid list number, width number, start number) ? number`
- `countRegions(grid list number, width number) ? number`
- `fillDist(grid list number, width number, start number) ? list number`
- `reachable(grid list number, width number, start number) ? number`
- `boundary(grid list number, width number, start number) ? list number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/gameflood` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : floodFill, conn

`import "std/c/gameflood"` · `use gameflood`

- `floodFill(grid list number, width number, start number, newVal number) ? list number`
- `connectedRegion(grid list number, width number, start number) ? number`
- `countRegions(grid list number, width number) ? number`
- `fillDist(grid list number, width number, start number) ? list number`
- `reachable(grid list number, width number, start number) ? number`
- `boundary(grid list number, width number, start number) ? list number`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/gameflood"
use gameflood
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? floodFill(grid list number, width number, start number, newVal number) ? list
? connectedRegion(grid list number, width number, start number) ? number
? countRegions(grid list number, width number) ? number
? fillDist(grid list number, width number, start number) ? list
? reachable(grid list number, width number, start number) ? number
? boundary(grid list number, width number, start number) ? list
```

4. Exemples d'utilisation

```
import "std/c/gameflood"
use gameflood

create result = floodFill(/* args */)
say result

create result = connectedRegion(/* args */)
say result

create result = countRegions(/* args */)
say result

create result = fillDist(/* args */)
say result

create result = reachable(/* args */)
say result

create result = boundary(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/gameflood"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module gameflood

function floodFill(grid list number, width number, start number, newVal number) returns list number
```

```
end

function connectedRegion(grid list number, width number, start number) returns number
end

function countRegions(grid list number, width number) returns number
end

function fillDist(grid list number, width number, start number) returns list number
end

function reachable(grid list number, width number, start number) returns number
end

function boundary(grid list number, width number, start number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/gameflood` (tier complex, category complex). Exports 6 function(s): floodFill, connectedRegion,

`import "std/c/gameflood"` · `use gameflood`

- `floodFill(grid list number, width number, start number, newVal number) ? list number`
- `connectedRegion(grid list number, width number, start number) ? number`
- `countRegions(grid list number, width number) ? number`
- `fillDist(grid list number, width number, start number) ? list number`
- `reachable(grid list number, width number, start number) ? number`
- `boundary(grid list number, width number, start number) ? list number`

2. Installation & import

Add the module to your program:

```
import "std/c/gameflood"
use gameflood
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? floodFill(grid list number, width number, start number, newVal number) ? list
? connectedRegion(grid list number, width number, start number) ? number
? countRegions(grid list number, width number) ? number
? fillDist(grid list number, width number, start number) ? list
? reachable(grid list number, width number, start number) ? number
? boundary(grid list number, width number, start number) ? list
```

4. Usage examples

```
import "std/c/gameflood"
use gameflood

create result = floodFill(/* args */)
say result

create result = connectedRegion(/* args */)
say result

create result = countRegions(/* args */)
say result

create result = fillDist(/* args */)
say result

create result = reachable(/* args */)
say result

create result = boundary(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/c/gameflood"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module gameflood

function floodFill(grid list number, width number, start number, newVal number) returns list number
```

```
end

function connectedRegion(grid list number, width number, start number) returns number
end

function countRegions(grid list number, width number) returns number
end

function fillDist(grid list number, width number, start number) returns list number
end

function reachable(grid list number, width number, start number) returns number
end

function boundary(grid list number, width number, start number) returns list number
end
end
```